

Objectifs. Concevoir une application client/serveur conforme à une version restreinte du protocole HTTP (Hypertext Transfer Protocol).

Rappel. HTTP est un protocole applicatif client-serveur basé sur le protocole de transport TCP. Il permet la consultation des ressources du Web identifiées par une URL. Le client (un navigateur) :

1. récupère une ressource, i.e. analyse l'URL, formule une requête et réceptionne la ressource;
2. interprète l'HTML, i.e. récupère les fichiers associés (images, feuille de style, script) et affiche une page web.

Le serveur Web écoute les requêtes, vérifie leur validité et répond avec une ressource ou un message d'erreur. Le protocole HTTP est un protocole sans connexion et sans état (*stateless*), chaque requête étant traitée indépendamment des précédentes.

Testez

Le but de cette partie est de tester un serveur HTTP.

Exercice 1 : naviguez

Les méthodes des requêtes les plus fréquentes sont :

Méthode	Signification
GET	Récupère une ressource
POST	Ajoute une ressource
HEAD	Récupère l'en-tête de la réponse
TRACE	Récupère la requête reçue par le serveur
OPTIONS	Récupère les méthodes supportées par le serveur

Par exemple, une requête se présente sous la forme

GET <url> HTTP/<version>

suivie d'une ligne vide et éventuellement d'informations complémentaires, sous la forme de couples clé : valeur (un par ligne) pour préciser l'encodage des données émises, le format de fichier accepté par le client, etc.

Une réponse du serveur se présente sous la forme

HTTP/<version> <code> <message>

suivie d'une ligne vide, puis éventuellement d'un contenu. Les codes de réponse les plus fréquents sont :

Code	Signification
2xx	Succès
3xx	Redirection
4xx	Erreur du client
5xx	Erreur du serveur

Q1. Dans votre navigateur favori, affichez les requêtes HTTP émises et les réponses reçues^{*†}. Consultez l'URL <http://perdu.com>. Quels sont les messages associés aux codes 200, 404, 418 et 502 ?

*. Mozilla Firefox : Paramètres → Outils supplémentaires/Outils de développement web → Réseau

†. Google Chrome : Paramètres → Plus d'outils/Outils de développement → Network

Exercice 2 : susurrez à l'oreille du serveur

Telnet (**te**letype **net**work) est un protocole applicatif client-serveur basé sur le protocole de transport TCP. Il permet de communiquer avec un système distant (serveur, routeur) en échangeant des lignes de texte.

Q1. Utilisez la commande `telnet perdu.com` pour envoyer les requêtes suivantes[‡] :

```
GET /\r\n
GET / HTTP/1.1\r\n\r\n
GET / HTTP/1.1\r\nHost:perdu.com\r\n\r\n
```

Pour chaque requête, expliquez la réponse obtenue. À quoi sert le champ Host ?

Implémentez

Le but de cette partie est de créer un serveur HTTP qui pourra répondre à des requêtes GET.

Exercice 3 : architecture

Le serveur doit pouvoir traiter les requêtes de plusieurs clients en parallèle.

Q1. Commencez par créer une arborescence de fichiers regroupant plusieurs pages web statiques, organisées dans différents sous-dossiers et reliées entre elles par des hyperliens.

Q2. En vous inspirant des TPs RESO, mettez en place l'architecture du serveur. Dans un premier temps, le serveur affiche uniquement les requêtes reçues sur sa sortie standard, sans effectuer de vérification de leur format. Veillez également à gérer les exceptions.

Q3. Testez votre serveur avec la commande `telnet localhost:6666`, puis avec votre navigateur.

Exercice 4 : décoder des requêtes HTTP

Le serveur doit pouvoir traiter les requêtes GET. Le serveur répond aux autres méthodes HTTP par une erreur 405 (méthode non prise en charge). Dans cet exercice, vous écrivez les méthodes qui réceptionnent et analysent des requêtes. Pour rappel, le format d'une requête GET est le suivant :

```
GET http://localhost:6666/foo.html HTTP/<version>\r\n
\r\n
foo:bar\r\n
bar:foo\r\n
Host:foobar\r\n
\r\n
Un message que le serveur ignore\r\n
\r\n
```

Q1. Écrivez une méthode qui reçoit une requête HTTP et la stocke sous forme de chaîne de caractères.

Q2. Écrivez une méthode^{*} qui vérifie si une requête HTTP est bien formée et retourne un code adapté : 400 si la requête est incorrecte, 405 si la méthode n'est pas supportée, ou 200 si tout est correct.

Exercice 5 : réponses du serveur - les erreurs

Pour rappel, le format des réponses du serveur est le suivant :

[‡]. Vérifiez que la commande `telnet` fonctionne correctement dans votre terminal (sous Windows, il peut être nécessaire de l'activer depuis le panneau de configuration).

^{*}. Pour l'analyse des chaînes de caractères, vous pouvez utiliser le paquetage `java.util.regex`.

```
HTTP/<version> <code> <message>\r\n
foo:bar\r\n
bar:foo\r\n
\r\n
Le contenu du fichier quand la requête aboutit\r\n
\r\n
```

Q1. Écrivez une méthode (surchargée) permettant de générer l'en-tête d'une réponse HTTP. Cette méthode devra accepter en argument soit uniquement un code HTTP, soit un code HTTP accompagné d'une taille de contenu. L'en-tête produit correspond à l'ensemble des lignes situées avant la première ligne vide. Il devra comporter les éléments suivants :

- la ligne de statut, composée du code et du message associés[†]. Les associations code–message pourront être stockées dans une table de correspondance. En cas de code inconnu, la méthode renverra une erreur 500 Internal Server Error;
- le champ Date: suivi de la date courante au format lun., 24 nov. 2025 09:45:39 CET;
- le champ Server: <nom_de_votre_serveur>;
- le champ Connection: close, indiquant que la connexion est fermée après le traitement de la requête (absence de mécanisme keep-alive);
- le champ Content-Length: <taille> (exprimée en octets), uniquement si l'argument taille a été fourni;
- le champ Content-Type: <type> dont la valeur sera, dans un premier temps, systématiquement text/html;
- une ligne vide terminant l'en-tête.

Q2. Écrivez une méthode qui génère la réponse du serveur en cas d'erreur. Celle-ci devra compléter l'en-tête produit par la méthode précédente en y ajoutant un message adapté (par exemple le code HTTP associé à la page d'erreur), suivi d'une ligne vide. Comme précédemment, vous pouvez utiliser un dictionnaire pour associer chaque code d'erreur à son message.

Exercice 6 : réponses du serveur - les autres

Cet exercice vous demande de prendre en compte plusieurs cas particuliers.

- Si le chemin indiqué dans la requête ne correspond à aucun fichier, la méthode doit renvoyer une erreur 404 Not Found.
- Si le chemin indiqué mène à un dossier et non à un fichier, la méthode doit renvoyer le chemin vers le fichier /index.html présent dans ce dossier. Par exemple, si //chemin/du/site/chemin/de/la/requete est un dossier, la méthode devra renvoyer //chemin/du/site/chemin/de/la/requete/index.html.
- Le champ de la requête peut contenir des éléments parasites. Il faut supprimer tout ce qui suit un événement ? (inclus), et remplacer chaque séquence %xx par le caractère ASCII correspondant.

Q1. Écrivez une méthode qui prend en entrée une requête et construit le chemin vers le fichier demandé. Par exemple, si votre serveur est situé dans le répertoire /chemin/du/site/ et que l'adresse fournie par la requête est //chemin/de/la/requete, la méthode devra renvoyer //chemin/du/site/chemin/de/la/requete.

Q2. Écrivez (enfin !) une méthode qui génère la réponse du serveur lorsqu'aucune erreur n'est détectée. Vous disposez alors d'un serveur fonctionnel. Testez-le avec votre navigateur !

Exercice 7 : virtualiser pour aller plus loin

À ce stade, votre serveur gère un seul site web situé sur votre machine, dans le dossier que vous avez choisi, par exemple /chemin/du/site. Il est toutefois courant qu'une même machine héberge plusieurs sites web.

Q1. Modifiez votre serveur afin qu'il prenne en charge deux sites web (placés dans deux dossiers distincts). Pour cela, appuyez-vous sur le champ Host: afin d'identifier le site auquel le client souhaite accéder. Si ce champ est absent, le serveur devra renvoyer une erreur.

[†]. Exemple : 404 → Not Found